

---

# 1KULTE

*découvrir le patrimoine, autrement.*

---

## DOSSIER TECHNIQUE DE CONCEPTION

*Spécifications techniques et méthodes de testing*

Mai 2026

# Sommaire

|                                               |    |
|-----------------------------------------------|----|
| Sommaire .....                                | 2  |
| 1. Introduction et contexte .....             | 4  |
| 1.1 Présentation du projet .....              | 4  |
| 1.2 Objectifs du dossier .....                | 4  |
| 1.3 Périmètre fonctionnel .....               | 4  |
| 2. Architecture générale.....                 | 5  |
| 2.1 Vue d'ensemble .....                      | 5  |
| 2.2 Stack par couche .....                    | 5  |
| 2.3 Modules backend .....                     | 6  |
| 2.4 Communication mobile – API .....          | 6  |
| 3. Technologies utilisées .....               | 7  |
| 3.1 Backend NestJS.....                       | 7  |
| 3.1.1 Validation globale des entrées .....    | 7  |
| 3.1.2 Persistance avec TypeORM .....          | 8  |
| 3.1.3 Stockage de fichiers MinIO .....        | 8  |
| 3.2 Application mobile .....                  | 8  |
| 3.2.1 Structure du projet mobile .....        | 9  |
| 3.3 Infrastructure et exécution .....         | 9  |
| 4. Mesures de sécurité .....                  | 10 |
| 4.1 Authentification et autorisation .....    | 10 |
| 4.2 Stockage sécurisé des mots de passe ..... | 10 |
| 4.3 Validation et nettoyage des entrées ..... | 10 |
| 4.4 Protection des données utilisateur .....  | 11 |
| 4.5 Cloisonnement réseau.....                 | 11 |
| 4.6 Sécurité du transport .....               | 11 |
| 4.7 Gestion des secrets .....                 | 11 |
| 4.8 Gestion des erreurs côté client .....     | 11 |
| 5. Méthodologie de testing .....              | 12 |
| 5.1 Stratégie globale.....                    | 12 |
| 5.2 Outillage .....                           | 12 |
| 5.3 Tests unitaires .....                     | 12 |
| 5.3.1 Exemple : test du login .....           | 13 |
| 5.4 Tests de bout en bout.....                | 13 |
| 5.5 Tests manuels via Bruno.....              | 13 |
| 5.6 Couverture et exécution.....              | 13 |

|                                                                  |    |
|------------------------------------------------------------------|----|
| 5.7 Intégration continue.....                                    | 14 |
| 6. Livraison continue (CI/CD).....                               | 15 |
| 6.1 Workflow GitHub Actions.....                                 | 15 |
| 6.2 Étapes du pipeline .....                                     | 15 |
| 6.3 Stratégie de minimisation de l'interruption de service ..... | 15 |
| 6.4 Migrations de la base de données.....                        | 15 |
| 7. Conclusion .....                                              | 16 |

# 1. Introduction et contexte

## 1.1 Présentation du projet

1Kulte est une application mobile dédiée à la découverte du patrimoine culturel et touristique. Elle propose des parcours thématiques (trails) associés à des lieux culturels, complétés par des questionnaires à choix multiples (QCM) gamifiés. Chaque réponse correcte rapporte des points, convertibles en récompenses via une boutique intégrée (panier, modes de livraison, points relais, cagnotte de fidélité).

→ L'application combine découverte culturelle, gamification et fidélisation au sein d'un même parcours utilisateur.

## 1.2 Objectifs du dossier

Ce document a pour objectifs de :

- présenter l'architecture technique générale de la solution ;
- décrire les technologies utilisées côté backend, mobile et infrastructure ;
- formaliser les mesures de sécurité mises en place ;
- documenter la méthodologie de tests appliquée au projet ;
- décrire la chaîne de livraison continue (CI/CD).

## 1.3 Périmètre fonctionnel

L'application couvre les domaines fonctionnels suivants :

- Gestion des utilisateurs (inscription, authentification, profil, photo de profil).
- Catalogue de lieux culturels avec géolocalisation, type, photos et popularité.
- Gestion de parcours (trails) avec QR code généré côté serveur.
- QCM associés aux trails avec scoring, gain de points, historique et progression.
- Favoris et recommandations personnalisées.
- Boutique de récompenses (panier, livraison, points relais, cagnotte).

## 2. Architecture générale

### 2.1 Vue d'ensemble

La solution s'articule autour de trois composants principaux :

- une application mobile multiplateforme (Expo / React Native) destinée aux utilisateurs finaux
- une API REST développée avec NestJS, qui expose les endpoints métier et gère la persistance
- une infrastructure conteneurisée (Docker) déployée sur un VPS et exposée via Traefik / Let's Encrypt.

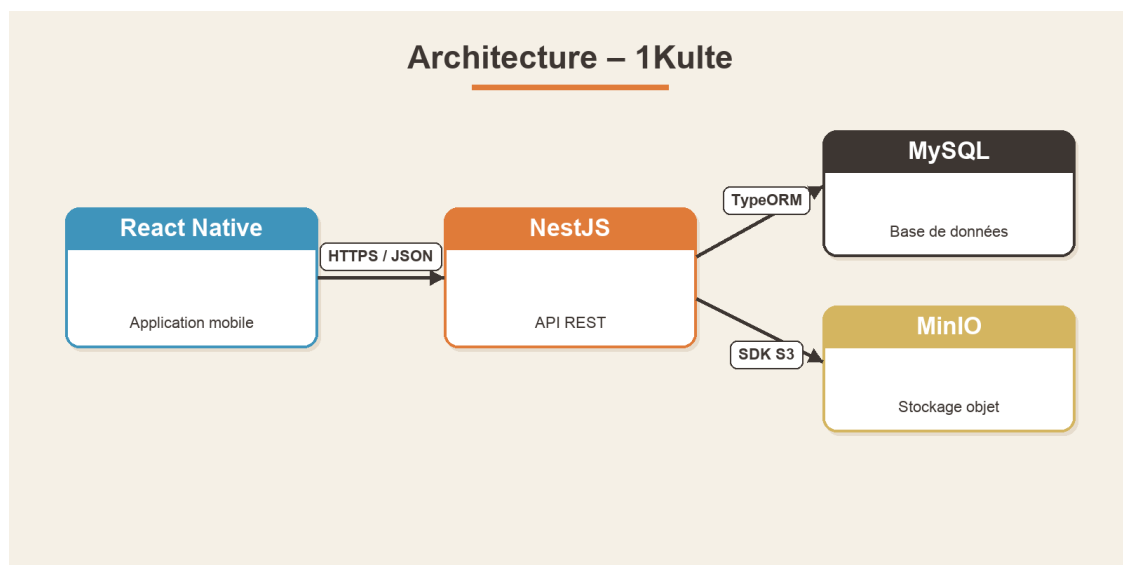


Figure 1 — Architecture : React Native consomme l'API NestJS, qui accède à MySQL et à MinIO.

### 2.2 Stack par couche

| Couche               | Technologies                                            |
|----------------------|---------------------------------------------------------|
| Application mobile   | React Native 0.81, Expo 54, expo-router, TypeScript 5.9 |
| API backend          | NestJS 11, TypeScript 5.7, runtime Bun, Express         |
| Base de données      | MySQL 8.0 via TypeORM 0.3                               |
| Stockage de fichiers | MinIO (compatible S3)                                   |
| Authentification     | JWT (@nestjs/jwt), bcrypt                               |
| Validation           | class-validator, class-transformer                      |
| Documentation API    | Collection Bruno (server/bruno/)                        |

|               |                                             |
|---------------|---------------------------------------------|
| Tests         | Jest 30, Supertest, ts-jest                 |
| CI/CD         | GitHub Actions, déploiement SSH automatique |
| Reverse proxy | Traefik v2 + Let's Encrypt                  |

## 2.3 Modules backend

L'API NestJS est organisée en modules métier ; chacun se compose d'un controller, d'un service, d'un repository et d'entités TypeORM :

- UsersModule : authentification, profil, photo de profil, cagnotte de points.
- CulturalPlaceModule : lieux culturels, classement par popularité, recommandations.
- CulturalPlacePictureModule : photos associées aux lieux.
- FavoriteModule : favoris d'un utilisateur.
- TrailModule : parcours et génération de QR code (data URL + image PNG).
- QcmModule : questions, réponses, soumission, progression, historique.
- RewardModule : récompenses, panier, livraison, points relais, cagnotte.
- Shares : modules transverses MinioModule (stockage) et CryptModule (bcrypt).

## 2.4 Communication mobile – API

L'application mobile communique exclusivement avec l'API via le client services/api.ts. Ce client centralise les appels HTTP fetch avec :

- un timeout (15 secondes) géré par AbortController ;
- un parsing JSON sécurisé (safeJsonParse) ;
- un mapping des codes HTTP vers des messages d'erreur en français ;
- une typologie d'erreurs métier (NETWORK, TIMEOUT, SERVER, CLIENT, VALIDATION).

## 3. Technologies utilisées

### 3.1 Backend NestJS

Le backend repose sur NestJS 11, un framework Node.js modulaire bâti sur Express, exécuté en production avec le runtime Bun.

| Dépendance                          | Rôle                                                   |
|-------------------------------------|--------------------------------------------------------|
| @nestjs/common,<br>@nestjs/core     | Cœur du framework (DI, modules, controllers).          |
| @nestjs/config                      | Lecture centralisée de la configuration via .env.      |
| @nestjs/typeorm + typeorm           | ORM relationnel, migrations versionnées.               |
| @nestjs/jwt                         | Émission et vérification des JSON Web Tokens.          |
| @nestjs/platform-express            | Adaptateur HTTP Express (multer pour le multipart).    |
| bcrypt                              | Hachage et vérification des mots de passe (10 rounds). |
| class-validator / class-transformer | Validation déclarative des DTO.                        |
| minio                               | Client SDK pour le stockage objet (S3-like).           |
| mysql2                              | Driver MySQL natif.                                    |
| qrcode                              | Génération des QR codes pour les trails.               |

#### 3.1.1 Validation globale des entrées

Dans main.ts, un ValidationPipe global applique trois règles strictes :

```
app.useGlobalPipes(new ValidationPipe({
  whitelist: true,
  forbidNonWhitelisted: true,
  transform: true,
}));
```

Cette configuration garantit que :

- **whitelist** : seules les propriétés présentes dans le DTO sont conservées ;
- **forbidNonWhitelisted** : toute propriété non déclarée est rejetée (HTTP 400) ;
- **transform** : les types primitifs et les dates sont convertis automatiquement.

### 3.1.2 Persistance avec TypeORM

L'accès aux données est centralisé via `data/datasource.ts` et le module `TypeOrmModule.forRootAsync`. Le schéma est versionné par 14 migrations situées dans `server/src/migrations/`. Les options `synchronize: false` et `migrationsRun: false` garantissent qu'aucune modification de schéma n'est appliquée automatiquement : seule la commande `migration:run:prod`, invoquée explicitement par le pipeline de déploiement, peut faire évoluer la base.

### 3.1.3 Stockage de fichiers MinIO

Les fichiers utilisateurs (photos de profil, images de récompenses, photos de lieux, images de questions QCM) sont stockés sur un serveur MinIO compatible S3. Le service `MinioService` :

- crée automatiquement le bucket au démarrage (`onModuleInit`) ;
- applique une politique publique en lecture seule sur les fichiers (`s3:GetObject`) ;
- nettoie les noms de fichiers (suppression des accents et des caractères spéciaux) ;
- préfixe chaque fichier d'un timestamp afin d'éviter les collisions ;
- expose une URL publique via la variable `MINIO_PUBLIC_URL`.

## 3.2 Application mobile

L'application mobile est développée en React Native 0.81 via Expo 54, avec un routage par fichiers fourni par expo-router 6. La gestion de session repose sur `AsyncStorage`.

| Dépendance                                     | Rôle                                                 |
|------------------------------------------------|------------------------------------------------------|
| expo, expo-router                              | Framework et routage fichier-driven (app/).          |
| expo-camera                                    | Lecture des QR codes pour démarrer un trail.         |
| expo-image-picker                              | Sélection de la photo de profil.                     |
| expo-linking, expo-web-browser                 | Liens profonds et ouverture web.                     |
| react-native-maps                              | Carte interactive des lieux culturels.               |
| @react-native-async-storage/async-storage      | Persistance locale du token JWT et de l'utilisateur. |
| react-native-reanimated, react-native-worklets | Animations performantes.                             |

### 3.2.1 Structure du projet mobile

- app/ : écrans organisés par expo-router (login, register, welcome, modal, etc.).
- app/(tabs)/ : navigation principale (index, rewards, trails, search, profile).
- app/cultural-place/[id].tsx et app/reward/[id].tsx : routes dynamiques.
- services/ : api.ts (client HTTP), storage.ts (persistance), errorHandler.ts.
- components/ et constants/ : composants partagés et tokens d'interface.

## 3.3 Infrastructure et exécution

L'API et ses dépendances sont conteneurisées via Docker Compose :

- api : NestJS compilé, exécuté par Bun, exposé par Traefik en HTTPS ;
- mysql : MySQL 8.0 avec healthcheck mysqladmin ping ;
- minio : stockage objet exposé sur deux domaines (API S3 et console) ;
- adminer : interface d'administration MySQL.

Le reverse proxy Traefik gère le TLS via Let's Encrypt ainsi que les redirections HTTP → HTTPS pour les trois services exposés (api-1kulte.maillet.bzh, minio-1kulte.maillet.bzh, adminer-1kulte.maillet.bzh).

## 4. Mesures de sécurité

### 4.1 Authentification et autorisation

Le système d'authentification repose sur des JSON Web Tokens signés en HS256 :

- le secret de signature est lu depuis la variable d'environnement JWT\_SECRET (jamais committée) ;
- la durée de vie d'un token est fixée à 7 jours ;
- la vérification est centralisée dans AuthGuard (extraction du header Authorization: Bearer <token>, validation via jwtService.verifyAsync, injection du payload dans request.user) ;
- toute exception lève une UnauthorizedException (HTTP 401) ;
- le guard est appliqué via @UseGuards(AuthGuard) sur tous les endpoints sensibles.

```
@UseGuards (AuthGuard)
@Post ('cart/checkout')
checkoutCart (@Req() req: AuthenticatedRequest) {
  const userId = req.user.user.id;
  return this.rewardService.checkoutCart (userId);
}
```

### 4.2 Stockage sécurisé des mots de passe

Le service CryptService hache les mots de passe avec bcrypt (10 rounds) :

- aucun mot de passe en clair n'est jamais persisté ni renvoyé par l'API ;
- la colonne password de l'entité User est annotée select: false, ce qui exclut sa lecture par défaut ;
- la comparaison s'effectue uniquement en mémoire via bcrypt.compare.

```
async hashPassword(password: string): Promise<string> {
  return bcrypt.hash(password, 10);
}
```

### 4.3 Validation et nettoyage des entrées

- DTO décorés avec class-validator (@IsEmail, @MinLength, @MaxLength, @IsBoolean, @IsDateString).
- ValidationPipe global avec forbidNonWhitelisted: true qui rejette toute propriété non déclarée.
- Nettoyage des noms de fichiers (suppression des accents, restriction à [a-z0-9-], déduplication des tirets).
- Uploads multipart constraints par FileInterceptor (champ image ou file).

## 4.4 Protection des données utilisateur

- Les routes de lecture et de mise à jour du profil sont protégées par JWT.
- L'entité User ne stocke qu'un minimum de données personnelles.
- Les anciennes photos de profil sont supprimées de MinIO avant tout nouvel enregistrement.

## 4.5 Cloisonnement réseau

La configuration docker-compose.prod.yml organise deux réseaux :

- traefik (externe) : seuls les services publics y sont rattachés (api, minio, adminer) ;
- 1kulte-internal : réseau privé entre l'API et MySQL / MinIO. MySQL n'expose aucun port sur l'hôte.

**MySQL n'est jamais joignable depuis l'extérieur du VPS : seul le conteneur API y a accès.**

## 4.6 Sécurité du transport

- Tous les domaines publics sont servis exclusivement en HTTPS via Traefik.
- Les certificats sont gérés automatiquement par Let's Encrypt.
- Un middleware Traefik force la redirection HTTP → HTTPS de manière permanente (301).

## 4.7 Gestion des secrets

- Tous les secrets (mot de passe BDD, JWT, clés MinIO) sont injectés via variables d'environnement.
- Les fichiers .env sont ignorés par Git.
- Le déploiement CI/CD utilise les GitHub Actions Secrets pour l'accès SSH (VPS\_HOST, VPS\_USERNAME, VPS\_SSH\_KEY, VPS\_PORT).

## 4.8 Gestion des erreurs côté client

L'application mobile applique une stratégie défensive d'erreurs via services/errorHandler.ts :

- typologie d'erreurs explicite (NETWORK, TIMEOUT, SERVER, CLIENT, VALIDATION, UNKNOWN) ;
- messages utilisateurs en français, sans divulguer la pile technique ;
- détails techniques journalisés uniquement en mode développement (\_\_DEV\_\_) ;
- timeout client (15 s) qui interrompt les requêtes pendantes.

## 5. Méthodologie de testing

### 5.1 Stratégie globale

La stratégie de tests retenue suit une pyramide à deux étages : une base solide de tests unitaires qui couvre chaque service et chaque controller, complétée par des tests de bout en bout qui démarrent une instance Nest réelle. L'objectif est d'assurer la non-régression à chaque évolution de l'API tout en gardant un cycle de retour court.

→ 16 fichiers \*.spec.ts couvrent l'intégralité des services et des controllers métier.

### 5.2 Outillage

| Outil           | Rôle                                                              |
|-----------------|-------------------------------------------------------------------|
| Jest 30         | Framework de tests, mocks, assertions.                            |
| ts-jest         | Compilation TypeScript à la volée pendant les tests.              |
| @nestjs/testing | Création de modules de test isolés avec injection de dépendances. |
| supertest       | Tests HTTP de bout en bout sur l'application Nest.                |
| Bun             | Exécution des scripts (bun run test, bun run test:e2e).           |

### 5.3 Tests unitaires

Chaque module backend dispose de fichiers \*.spec.ts colocalisés à côté du code source. Le pattern retenu est :

- un fichier de test par controller et un fichier de test par service ;
- mocking systématique des dépendances via `jest.Mocked<T>` et `useValue: { ... jest.fn() }` ;
- couverture des cas nominaux et des cas d'erreur (`NotFoundException`, `BadRequestException`, branches d'autorisation).

Modules couverts :

- user : controller + service (authentification, hachage, login, mise à jour du profil, upload de photo) ;
- cultural-place : controller + service (populaires, recommandations, CRUD) ;
- cultural-place-picture : upload, photo principale, suppression ;
- favorite : ajout, suppression, listage ;
- trail : CRUD + génération du QR code (`getQRCode`, `getQRCodeImage`) ;

- qcm : questions, réponses, soumission, historique, progression ;
- reward : CRUD, panier, livraison, cagnotte, points relais, achat ;
- reward/dto/get-nearby-relay-points.dto.spec.ts : validation pure du DTO.

### 5.3.1 Exemple : test du login

Le test `UserService > login` illustre la stratégie de mocking :

```
userRepository.findByEmail.mockResolvedValue(mockUser);
cryptService.comparePasswords.mockResolvedValue(true);
jwtService.sign.mockReturnValue('jwt-token');

const result = await service.login({ email, password });

expect(result.token).toBe('jwt-token');
```

Trois scénarios sont systématiquement couverts : identifiants valides, email inconnu, mot de passe incorrect. La même approche est appliquée aux uploads de photos (cas nominal, suppression de l'ancienne photo, échec de suppression non bloquant, utilisateur inconnu).

## 5.4 Tests de bout en bout

Les tests E2E sont placés dans `server/test/` et configurés par `jest-e2e.json` (`testRegex .e2e-spec.ts$`). Ilsinstancient une application Nest réelle via `Test.createTestingModule` et utilisent `supertest` pour effectuer des requêtes HTTP sur l'`HttpServer` exposé.

## 5.5 Tests manuels via Bruno

La collection `server/bruno/` constitue la documentation exécutable de l'API. Elle est structurée en dossiers numérotés (00-App, 01-Users, 02-CulturalPlaces, ...) et tenue à jour à chaque évolution d'endpoint. Deux environnements sont disponibles :

- `local.bru` → `http://localhost:3000` pour le développement local ;
- `vps.bru` → `https://api-1kulte.maillet.bzh` pour la pré-production.

Le scénario `01-Users/04-Login.bru` injecte automatiquement le JWT obtenu dans `bru.setVar('jwt', token)` et `bru.setEnvVar('jwt', token)`, ce qui permet d'enchaîner les requêtes authentifiées sans manipulation manuelle.

## 5.6 Couverture et exécution

```
bun run test          # exécution unique
bun run test:watch    # mode watch pour le développement
bun run test:cov      # rapport de couverture dans coverage/
bun run test:e2e      # tests de bout en bout
```

La configuration Jest est inline dans package.json : la racine des tests est src/, le moduleNameMapper réécrit src/... pour permettre les imports absolus, et la couverture est collectée sur tout le code TypeScript.

## 5.7 Intégration continue

Le workflow GitHub Actions .github/workflows/deploy.yml exécute les tests à chaque push sur main qui touche server/\*\* :

- job test : bun install --frozen-lockfile puis bun run test (avec cache ~/.bun/install/cache) ;
- job deploy : exécuté uniquement si le job test réussit (déclaration needs: [test]).

**Aucun déploiement ne peut intervenir sur le VPS si un test unitaire échoue.**

## 6. Livraison continue (CI/CD)

### 6.1 Workflow GitHub Actions

Le déploiement est entièrement automatisé via le workflow `.github/workflows/deploy.yml` déclenché à chaque push sur main modifiant `server/**` ou le workflow lui-même.

### 6.2 Étapes du pipeline

- Checkout du code et installation de Bun via `oven-sh/setup-bun@v2`.
- Restauration du cache Bun (clé `bun-${hashFiles('server/bun.lock')}`).
- Installation des dépendances avec `bun install --frozen-lockfile`.
- Exécution de la suite de tests unitaires (`bun run test`).
- Connexion SSH au VPS (`appleboy/ssh-action`) avec les secrets `VPS_*`.
- Pull Git, build de l'image Docker, redémarrage des conteneurs avec `docker compose -p 1kulte-prod`.
- Healthcheck applicatif (30 tentatives, une par seconde) sur `http://localhost:3000`.
- Exécution des migrations TypeORM en production (`bun run migration:run:prod`).
- Nettoyage des images Docker obsolètes (`docker image prune -f`).

### 6.3 Stratégie de minimisation de l'interruption de service

Le pipeline construit l'image Docker avant de stopper les conteneurs existants, ce qui réduit la fenêtre d'indisponibilité. Le healthcheck attend jusqu'à 30 secondes que l'API redevienne joignable et abandonne en cas d'échec en affichant les 50 dernières lignes de logs (`docker logs 1kulte-api --tail 50`).

### 6.4 Migrations de la base de données

Les migrations TypeORM sont versionnées dans `server/src/migrations/` (14 migrations à ce jour, de `InitUserTable` jusqu'à `AddUseWalletDiscountToRewardCart`). Elles sont jouées explicitement en post-déploiement via `bun run migration:run:prod`. Les options `synchronize: false` et `migrationsRun: false`, au niveau de la connexion TypeORM, interdisent toute modification de schéma involontaire au démarrage de l'application.

## 7. Conclusion

---

L'application 1Kulte met en œuvre une architecture client / serveur moderne, conteneurisée et automatisée, reposant sur des technologies éprouvées (NestJS, TypeORM, MySQL, MinIO, React Native / Expo).

Les mesures de sécurité couvrent l'authentification (JWT 7 jours, bcrypt 10 rounds), la validation stricte des entrées (ValidationPipe global avec whitelist), le cloisonnement réseau Docker (MySQL non exposé publiquement), le chiffrement du transport (HTTPS via Let's Encrypt) et la gestion des secrets via variables d'environnement.

La méthodologie de tests combine des tests unitaires exhaustifs (16 fichiers \*.spec.ts couvrant tous les services et les controllers métier), une infrastructure de tests de bout en bout fondée sur Supertest, et une collection Bruno qui sert de documentation vivante de l'API. Le pipeline GitHub Actions exécute systématiquement les tests avant tout déploiement, ce qui garantit une mise en production fiable et tracée.

La séparation claire des modules NestJS et la centralisation des préoccupations transverses (CryptService, MinioService, AuthGuard) facilitent l'évolution du produit ainsi que sa maintenabilité à long terme.

**→ Stack moderne, sécurité « by design », tests exhaustifs : une base solide pour la croissance du produit.**